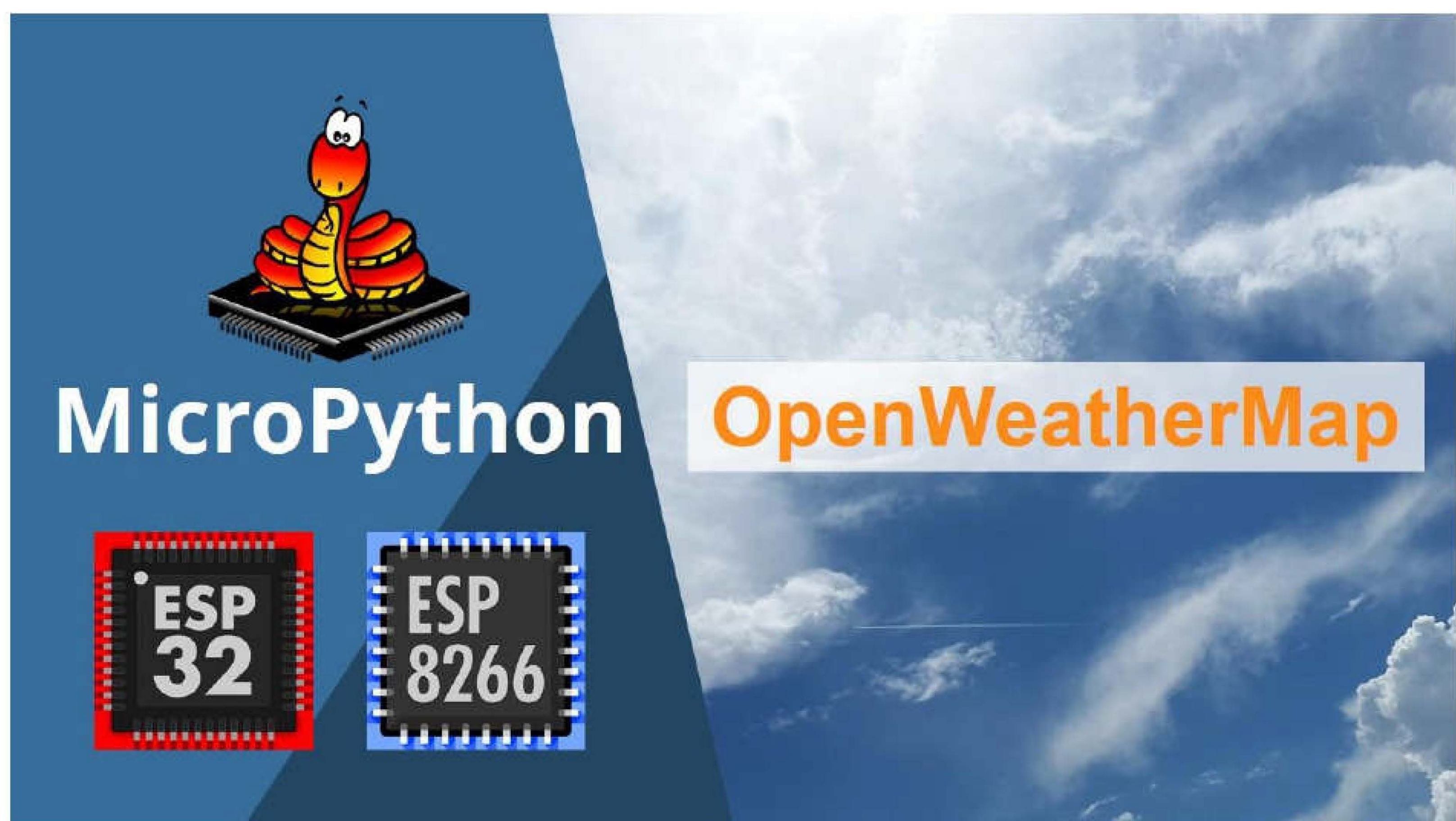


Getting Data Using an API



In this Unit you'll learn how to make API requests to access data. As an example, we'll use the OpenWeatherMap API. This API is free and provides lots of useful information about the weather in almost any location in the world.

Using the OpenWeatherMap API

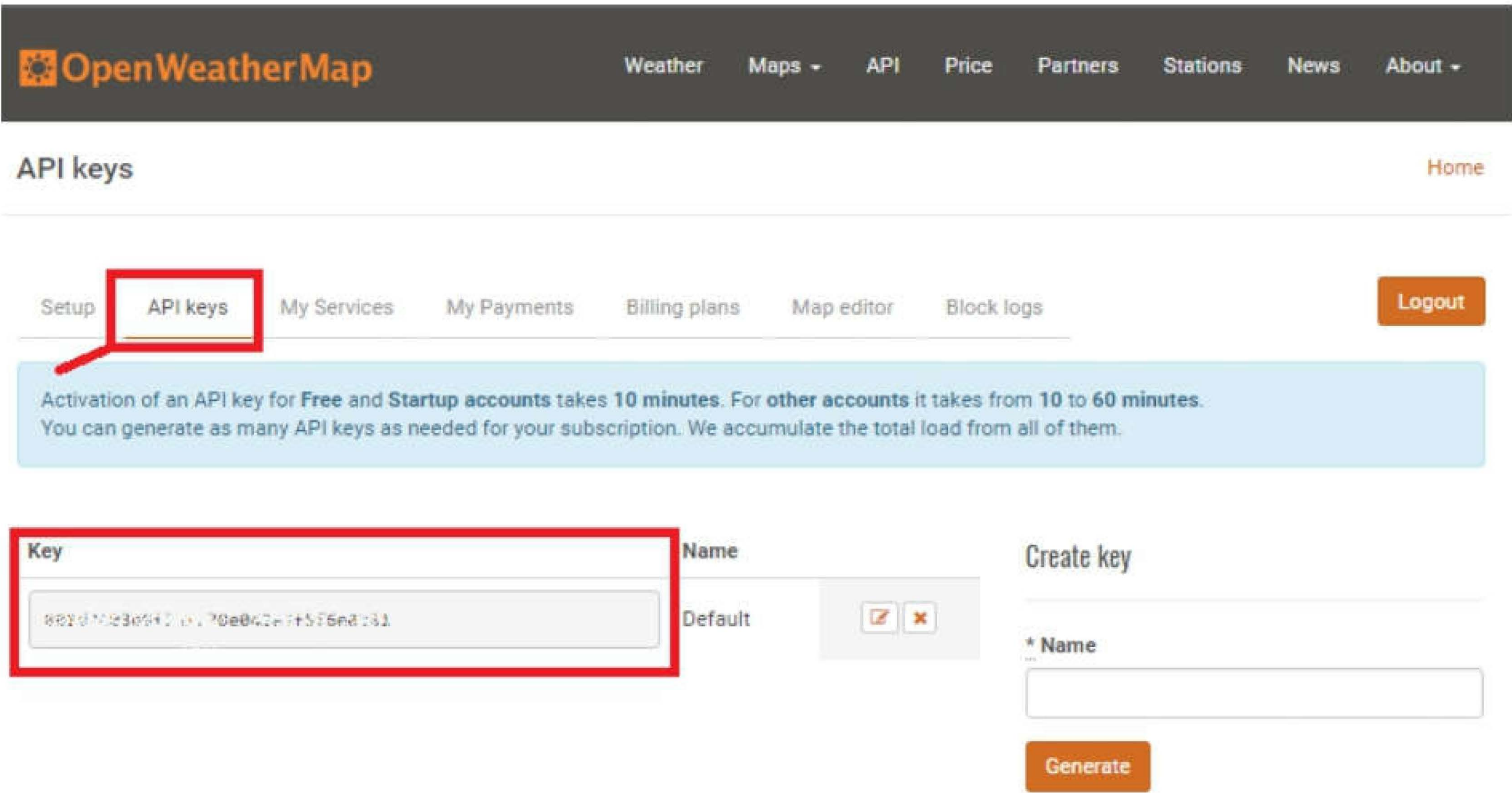
An application programming interface (API) is a set of functions written by software developers to enable anyone to use their data or services. For example, the **OpenWeatherMap** project ([https:// openweathermap.org/](https://openweathermap.org/)) has an API that enables users to request weather data using many different programming languages.



In this project, you'll use that API to request the day's weather forecast for your chosen location. Learning to use APIs is a great skill because it allows you access to a wide variety of constantly changing information, such as current stock prices, currency exchange rates, the latest news, traffic updates, tweets, and much more.

OpenWeatherMap’s free plan provides everything you need to complete this project. To use the API you need an API key, known as the APIID. To get the APIID:

- 1. Open a browser and go to <https://openweathermap.org/appid/>
- 2. Press the **Sign up** button and create a free account.
- 3. You’ll be presented with a dashboard that contains several tabs. Select the **API keys** tab, as shown in the following figure.



- 4. On the API keys tab, you’ll see a default key (highlighted in a red rectangle in figure above); this is a unique key you’ll need to pull information from the site. Copy and paste this key somewhere; you’ll need it in a moment.
- 5. To pull information on weather in your chosen location, enter the following URL:

```
http://api.openweathermap.org/data/2.5/weather?q=your_city,  
your_country_code&APPID=your_unique_API_key
```

Replace **your_city** with the city you want data for, **your_country_code** with the country code for that city, and **your_unique_API_key** with the unique API key from step 4. For example, the updated API URL for the city of Porto, Portugal, would be:

```
http://api.openweathermap.org/data/2.5/weather?q=Porto,  
PT&APPID=801d2603e9f2e1c70e042e4f5f6e0---
```


Note: API keys are unique to the user and shouldn't be shared with anyone.

6. Copy your URL into your browser, and the API will return a bunch of information corresponding to your local weather. We got the following information about the weather in Porto, Portugal, on the day we wrote this project.

```
{ "coord": { "lon": 8.61, "lat": 41.15 }, "weather": [ { "id": 803, "main": "Clouds", "description": "broken clouds", "icon": "04d" } ], "base": "stations", "main": { "temp": 285.67, "pressure": 1017, "humidity": 71, "temp_min": 285.15, "temp_max": 286.15 }, "visibility": 10000, "wind": { "speed": 5.7, "deg": 340 }, "clouds": { "all": 75 }, "dt": 1548781200, "sys": { "type": 1, "id": 6900, "message": 0.0036, "country": "PT", "sunrise": 1548748072, "sunset": 1548784072 }, "id": 2735943, "name": "Porto", "cod": 200 }
```

Next, you'll see how to organize this data with tabs and paragraphs to make it more readable.

Understanding JSON Syntax

As you can see, the weather data for your chosen location is stored in a specific way, with symbols like `{}` `[]` `:` `"` and `,`. This syntax is JavaScript Object Notation (JSON), a standard for exchanging data in a convenient way. In JSON syntax:

- Data is represented in name/value pairs.
- Each name is followed by a colon (`:`).
- Name/value pairs are separated with commas.
- Curly brackets hold objects.
- Square brackets hold arrays.

Below is the same data returned earlier but with spaces and indentation in a way that is easier to understand.

```
{
  "coord": {
    "lon": -8.61,
    "lat": 41.15
  },
  "weather": [ {
    "id": 803,
```



```
        "main": "Clouds",
        "description": "broken clouds",
        "icon": "04d"
    },
],
"base": "stations",
"main": {
    "temp": 285.67,
    "pressure": 1017,
    "humidity": 71,
    "temp_min": 285.15,
    "temp_max": 286.15
},
"visibility": 10000,
"wind": {
    "speed": 5.7,
    "deg": 340
},
"clouds": {
    "all": 75
},
"dt": 1548781200,
"sys": {
    "type": 1,
    "id": 6900,
    "message": 0.0036,
    "country": "PT",
    "sunrise": 1548748072,
    "sunset": 1548784072
},
"id": 2735943,
"name": "Porto",
"cod": 200
}
```

Now you can easily see all the useful information that the OpenWeatherMap API provides.

Making an API Request

After having a URL that returns your local weather data, we'll write a MicroPython script that returns temperature, humidity, pressure and wind for your chosen location. This information is then printed on the Shell.

boot.py

Copy the following script to the *boot.py* file. The goal of this project is to demonstrate how to decode JSON data from an API, so you can apply this concept to your own projects.

```
#import necessary libraries
import time

try:
    import urequests as requests
except:
    import requests

try:
    import ujson as json
except:
    import json

import network

import esp
esp.osdebug(None)

import gc
gc.collect()

ssid = 'REPLACE_WITH_YOUR_SSID'
password = 'REPLACE_WITH_YOUR_PASSWORD'

city = 'REPLACE_WITH_YOUR_CITY_NAME'
country_code = 'REPLACE_WITH_YOUR_2_LETTER_COUNTRY_CODE'

open_weather_map_api_key = 'REPLACE_WITH_YOUR_API_KEY'

station = network.WLAN(network.STA_IF)
```



```

station.active(True)
station.connect(ssid, password)

while station.isconnected() == False:
    pass

print('Connection successful')
print(station.ifconfig())

#set your unique OpenWeatherMap.org URL
open_weather_map_url =
'http://api.openweathermap.org/data/2.5/weather?q=' + city +
',' + country_code + '&APPID=' + open_weather_map_api_key

weather_data = requests.get(open_weather_map_url)
print(weather_data.json())

# Location (City and Country code)
location = 'Location: ' + weather_data.json().get('name') + '
- ' + weather_data.json().get('sys').get('country')
print(location)

# Weather Description
description = 'Description: ' +
weather_data.json().get('weather')[0].get('main')
print(description)

# Temperature
raw_temperature = weather_data.json().get('main').get('temp') - 273.15

# Temperature in Celsius
temperature = 'Temperature: ' + str(raw_temperature) + '*C'
#uncomment for temperature in Fahrenheit
#temperature = 'Temperature: ' +
str(raw_temperature*(9/5.0)+32) + '*F'
print(temperature)

# Pressure
pressure = 'Pressure: ' +
str(weather_data.json().get('main').get('pressure')) + 'hPa'
print(pressure)

# Humidity
humidity = 'Humidity: ' +
str(weather_data.json().get('main').get('humidity')) + '%'
print(humidity)

```



```
# Wind
wind = 'Wind: ' +
str(weather_data.json().get('wind').get('speed')) + 'mps ' +
str(weather_data.json().get('wind').get('deg')) + '*'
print(wind)
```

SOURCE CODE

https://github.com/RuiSantosdotme/ESP-MicroPython/blob/master/code/WiFi/HTTP_API_OpenWeatherMap/boot.py

Let's take a quick look on how the code works.

Importing libraries

You start by importing the necessary libraries:

```
import time

try:
    import urequests as requests
except:
    import requests

try:
    import ujson as json
except:
    import json

import network
```

You need the `urequests` and `ujson` library to make API requests and access the data. You need the `network` library to connect the ESP32/ESP8266 board to your router via Wi-Fi.

Entering your credentials

Then, you need to enter your network credentials:

```
ssid = 'REPLACE_WITH_YOUR_SSID'
password = 'REPLACE_WITH_YOUR_PASSWORD'
```


Enter the city you want to get data for, as well as the country code in the following variables:

```
city = 'REPLACE_WITH_YOUR_CITY_NAME'  
country_code = 'REPLACE_WITH_YOUR_2_LETTER_COUNTRY_CODE'
```

You also need to enter your unique API key on the following line:

```
open_weather_map_api_key = 'REPLACE_WITH_YOUR_API_KEY'
```

Connecting to the network

Then, we make the usual procedures to connect the ESP32/ESP8266 to your local network.

```
station = network.WLAN(network.STA_IF)  
  
station.active(True)  
station.connect(ssid, password)  
  
while station.isconnected() == False:  
    pass  
  
print('Connection successful')  
print(station.ifconfig())
```

Making the API request

Then, we create a variable called `open_weather_map_url` to hold the API URL with the credentials you've entered earlier: city, country code, and API key.

```
open_weather_map_url =  
'http://api.openweathermap.org/data/2.5/weather?q=' + city +  
, ' + country_code + '&APPID=' + open_weather_map_api_key
```

Finally, you make the API request as follows:

```
weather_data = requests.get(open_weather_map_url)
```

Basically, you create a variable called `weather_data` in which you store the data returned after the API request. To make an API request for information, you use the command `requests.get(your_url)`, with your URL as the argument.

Then, print the raw data on the Shell. To get the data, you first need to convert the `weather_data` variable to JSON with the `.json()` method.

```
print(weather_data.json())
```

After that, we have several lines of code that work in a similar way. These lines access specific pieces of information on the API response. For example, as seen in the “Understanding JSON syntax” section, inside the `'wind'`, you have `'speed'` and `'deg'`.

```
"wind": {
    "speed": 5.7,
    "deg": 340
```

To access speed:

```
weather_data.json().get('wind').get('speed')
```

Basically, you need to use the `get()` method to access specific data.

The same way, to access the location, you use:

```
weather_data.json().get('sys').get('country')
```

Demonstration

Copy the previous script to the ESP32/ESP8266 `boot.py` file. After uploading the code, press the EN/RESET button. It should print the weather data for your specific location.

```
Connection successful
('192.168.1.77', '255.255.255.0', '192.168.1.254', '192.168.1.254')
{'sys': {'message': 0.0033, 'country': 'PT', 'sunrise': 1548748049, 'sunset': 1548784058, 'id': 6900,
'type': 1}, 'name': 'Gondomar', 'main': {'pressure': 1017, 'humidity': 76, 'temp_min': 284.15,
'temp_max': 285.15, 'temp': 284.66}, 'cod': 200, 'base': 'stations', 'wind': {'speed': 3.6, 'deg':
340}, 'id': 2738925, 'clouds': {'all': 75}, 'coord': {'lon': -8.53, 'lat': 41.14}, 'dt': 1548784800,
'weather': [{'id': 803, 'icon': '04n', 'main': 'Clouds', 'description': 'broken clouds'}],
'visibility': 10000}
Location: Gondomar - PT
Description: Clouds
Temperature: 11.51°C
Pressure: 1017hPa
Humidity: 76%
Wind: 3.6mps 340°
```

You can also try to request data on other locations by changing the `city` and `country_code` variables.